

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 2(2)

Student Name: J Satya Srikar

Roll No.: A24126552259

Date: 18/8/26

1. TITLE

Student Profile Manager using JavaScript Class and DOM

2. AIM

Student Profile Manager using JavaScript Class and DOM

3. OBJECTIVE

The objectives of this observation task are:

- To study DOM element selection and interaction using vanilla JavaScript.
 - To capture user input datasets dynamically without triggering standard page refreshes.
 - To construct new HTML card structures in memory using `document.createElement()`.
 - To append new profile elements to the visual DOM tree dynamically using `appendChild()`.
 - To apply structural layout classes (e.g. containers, fields, buttons, profiles) dynamically.
-

4. THEORY

The Document Object Model (DOM) is an API exposing HTML layouts. JavaScript binds to this API using elements access methods. Form elements validation parameters block default execution chains via event overrides. In this application, inputs values (Name, Roll, Department, CGPA) are extracted upon submission, a new profile-card node is created dynamically, styled using local class selectors, and appended inside target layouts, representing runtime data updates in visual formats.

Theory of DOM Nodes Creation & Binding:

- **DOM Event Capturing:** Listening to submit actions via `addEventListener`. Execution processes are halted using `event.preventDefault()`.

- **Values extraction:** Read from input nodes using *document.getElementById().value*.
 - **Memory Node Allocation:** Instantiated using *document.createElement('div')*. Classes are appended using the *classList.add('profile-card')* method.
 - **HTML Template interpolation:** Injected inside element structures via the *innerHTML* property, structuring labels and records.
 - **DOM Insertion:** Handled using *appendChild()*, pushing nodes into the visual page tree.
-

5. ALGORITHM / PROCEDURE

1. Set up project folders: *Observation/ObTask2-(2)/src*, *Docs*, and *Screenshots*.
 2. Write *profile.html* structuring fields for Student profiles inputs.
 3. Create *style.css* defining backgrounds, borders, layouts alignments, card visual structures, and button hovers.
 4. Create *script.js*, select input nodes, and bind submit handlers.
 5. In script, block default submission loops, extract input values, and instantiate card containers.
 6. Inject html structures containing details into memory containers, and clear previous items.
 7. Append student details card back into page layout DOM nodes.
 8. Launch file in Chrome, run profile inputs test cases, and verify visual updates.
-

6. PROGRAM

Files: *profile.html* and *script.js*

profile.html (Complete) & script.js (Complete)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile Generator</title>
  <link rel="stylesheet" href="./style.css">
</head>

<body>

  <div class="container">
    <h1>Student Profile Generator</h1>

    <form id="studentForm">

      <label for="name">Name</label>
      <input type="text" id="name" placeholder="Enter your name" required>

      <label for="rollNo">Roll No</label>
      <input type="text" id="rollNo" placeholder="Enter your roll number" required>

      <label for="department">Department</label>
      <input type="text" id="department" placeholder="Enter your department" required>

      <label for="cgpa">CGPA</label>
      <input type="number" id="cgpa" step="0.01" min="0" max="10"
        placeholder="Enter your CGPA" required>

      <button type="submit">Generate Profile</button>

    </form>

    <!-- Profile will be generated here -->
    <div id="profileContainer"></div>

  </div>

  <script src="./script.js"></script>
</body>
</html>

// --- script.js ---
// Get the form from DOM
const form = document.getElementById("studentForm");

// Get the container where profile will be created
const profileContainer = document.getElementById("profileContainer");

// Listen for form submission
form.addEventListener("submit", function(event) {

  // Prevent page refresh
  event.preventDefault();

  // Get values from input fields
  const name = document.getElementById("name").value;
  const rollNo = document.getElementById("rollNo").value;
  const department = document.getElementById("department").value;
  const cgpa = document.getElementById("cgpa").value;

  // Create profile card
  const profileCard = document.createElement("div");

  // Add CSS class
  profileCard.classList.add("profile-card");

  // Add profile information
  profileCard.innerHTML = `
    <h2>Student Profile</h2>

    <p><strong>Name:</strong> ${name}</p>

    <p><strong>Roll No:</strong> ${rollNo}</p>

    <p><strong>Department:</strong> ${department}</p>

    <p><strong>CGPA:</strong> ${cgpa}</p>
  `;

  // Clear previous profile
  profileContainer.innerHTML = "";

  // Add new profile card to DOM
  profileContainer.appendChild(profileCard);
});
```

style.css (Complete)

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background: #f2f4f7;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.container {
  width: 400px;
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.15);
}

h1 {
  text-align: center;
  margin-bottom: 25px;
  color: #333;
}

label {
  display: block;
  margin-top: 15px;
  margin-bottom: 5px;
  font-weight: bold;
}

input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 15px;
}

button {
  width: 100%;
  margin-top: 25px;
  padding: 12px;
  border: none;
  border-radius: 6px;
  background: #2563eb;
  color: white;
  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background: #1d4ed8;
}

#profileContainer {
  margin-top: 30px;
}

.profile-card {
  padding: 20px;
  border-radius: 10px;
  background: #f8fafc;
  border: 1px solid #ddd;
}

.profile-card h2 {
  margin-bottom: 15px;
  color: #2563eb;
}

.profile-card p {
  margin: 10px 0;
  color: #333;
}
```

7. CODE EXPLANATION

DOM Selection / Statement	Explanation
<code>addEventListener("submit", function(event) { ... })</code>	Binds form submission handler, triggering JavaScript calculations upon submission.
<code>event.preventDefault()</code>	Hhalts default page reload sequences upon form submittals.
<code>document.getElementById("name").value</code>	Reads string value from text field input.
<code>document.createElement("div")</code>	Allocates a new HTML element node (div) in browser memory space.
<code>profileCard.classList.add("profile-card")</code>	Appends class name to node metadata, applying corresponding CSS configurations.
<code>profileCard.innerHTML = `...`</code>	Injects dynamic details variables into card HTML layout strings.
<code>profileContainer.innerHTML = ""</code>	Clears previously rendered student profile card nodes.
<code>profileContainer.appendChild(profileCard)</code>	Inserts card layout child node into target profile wrapper layout.

8. TEST CASES AND EXECUTION DATA

The following test executions verify DOM profile card generation inside Google Chrome:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Required Validations	Submit form with empty Name	Form blocked; browser validation prompt displays	Pass
TC-02	Profile Creation	Fill valid details and submit	Student details card is generated and appended below form	Pass
TC-03	Dynamic DOM Refresh	Submit new student details	Previous details card is cleared and replaced by new details	Pass
TC-04	Values Integrity	Inspect values inside DOM card	Name, Roll, Department, CGPA match exactly with input parameters	Pass

9. OUTPUT

Output: Rendered Student Profile Generator Webpage

Student Profile Generator

Name

J SATYA SRIKAR

Roll No

A24126552259

Department

CSM-B

CGPA

8.07

Generate Profile

Student Profile

Name: J SATYA SRIKAR

Roll No: A24126552259

Department: CSM-B

CGPA: 8.07

10. RESULT

Thus, student profile card generation utility (incorporating JavaScript form submissions listeners, element node creations, class assignments, and innerHTML integrations) was successfully implemented, executed, and verified.

Submission Package Structure:

```
Observation/ObTask2-(2)/
  ■■■ Docs/
  ■ ■■■ ObsWeek2(2).pdf
  ■■■ Screenshots/
  ■ ■■■ image.png
  ■■■ src/
  ■■■ profile.html
  ■■■ script.js
  ■■■ style.css
```